

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 6
по дисциплине «Анализ алгоритмов»

Тема Задача коммивояжёра

Студент Атаев К. А.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л., Строганов Д.В

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Основные понятия теории графов 5

1.2 Метод полного перебора 5

1.3 Муравьиный алгоритм 6

1.4 Вывод 7

2 Конструкторская часть **8**

2.1 Схемы алгоритмов 8

2.2 Трудоёмкости алгоритмов 13

2.2.1 Модель вычислений 13

2.2.2 Алгоритм полного перебора 13

2.2.3 Муравьиный алгоритм 14

2.2.4 Сравнение алгоритмов 15

3 Технологическая часть **17**

3.1 Средства реализации 17

3.2 Реализация алгоритмов 17

3.3 Демонстрация работы программы 17

3.4 Функциональные тесты 18

4 Исследовательская часть **19**

4.1 Технические характеристики 19

4.2 Результаты замеров 19

4.3 Параметризация 21

4.3.1 Рекомендации по выбору параметров 21

4.4 Вывод 22

ЗАКЛЮЧЕНИЕ **23**

ПРИЛОЖЕНИЕ А **25**

ВВЕДЕНИЕ

Целью работы является сравнительный анализ методов решения задачи коммивояжёра на основе полного перебора и муравьиного алгоритма для заданного варианта графа с выполнением параметризации муравьиного алгоритма по его основным параметрам.

Для достижения цели необходимо выполнить следующие задачи:

1) сформулировать постановку рассматриваемого варианта задачи коммивояжёра;

2) описать и реализовать метод полного перебора для поиска гамильтонова цикла с ограничением по длине;

3) описать и реализовать метод решения задачи на основе муравьиного алгоритма без элитных муравьёв и с необнуляемым феромоном;

4) выполнить параметризацию муравьиного алгоритма по трём его параметрам и оценить влияние параметров на качество получаемых решений относительно эталонного результата;

5) провести сравнительный анализ методов по полученному времени выполнения;

1 Аналитическая часть

Задача коммивояжёра заключается в поиске маршрута минимальной длины, проходящего по всем вершинам графа ровно один раз и возвращающегося в исходную вершину. Такой маршрут называется гамильтоновым циклом. В работе рассматривается неориентированный полносвязный граф, на рёбрах которого заданы затраты по времени. Дополнительно вводится ограничение на суммарную длину маршрута: допустимыми считаются только циклы с общей продолжительностью не более 80 условных дней. Общие сведения о задачах дискретной оптимизации и маршрутизации можно найти в [1].

1.1 Основные понятия теории графов

Графом называется пара $G = (V, E)$, где V — непустое конечное множество вершин, а E — множество рёбер, каждое из которых соединяет две вершины из V. Если каждому ребру (i, j) сопоставлено числовое значение c_{ij}, то говорят о взвешенном графе, а величина c_{ij} интерпретируется как стоимость или длина перехода между вершинами i и j.

Граф называется неориентированным, если ребро (i, j) не имеет направления, то есть переход из вершины i в вершину j эквивалентен переходу из j в i, и вес ребра удовлетворяет равенству $c_{ij} = c_{ji}$. Граф называется полносвязным (полным), если каждая пара различных вершин соединена ребром.

Маршрутом в графе называется последовательность вершин, в которой каждые две соседние вершины соединены ребром. Если маршрут начинается и заканчивается в одной и той же вершине и при этом каждая вершина графа встречается в нём ровно один раз, такой маршрут называется гамильтоновым циклом. В рассматриваемой работе используется взвешенный неориентированный полносвязный граф, на рёбрах которого заданы затраты по времени, а искомый маршрут представляет собой гамильтонов цикл с ограничением на суммарную длину.

1.2 Метод полного перебора

Метод полного перебора предполагает просмотр всех возможных перестановок вершин графа. В рассматриваемой постановке фиксируется начальная вершина (например, вершина с номером ноль), после чего перебираются все перестановки остальных вершин. Для каждой перестановки строится гамильтонов цикл с возвратом в исходную вершину и вычисляется его длина. Если длина превышает заданное ограничение, маршрут отбрасывается. Подробное изложение классических методов перебора и оценки их трудоёмкости приведено, в частности, в [1].

Преимущества метода:

— гарантированное нахождение оптимального по длине маршрута среди всех допустимых циклов;

— простота идеи и реализации;

Недостаток метода:

— невозможность применения для графов большого размера из-за экспоненциального роста времени выполнения;

Метод полного перебора используется в работе как эталон для оценки качества приближённых решений, получаемых вероятностными методами.

1.3 Муравьиный алгоритм

Муравьиный алгоритм относится к стохастическим методам оптимизации и основан на моделировании поведения муравьиной колонии при поиске кратчайших путей между гнездом и источниками пищи. Общее изложение идей муравьиных алгоритмов и их применения к задачам маршрутизации приведено, например, в [2]. В контексте задачи коммивояжёра каждый муравей строит маршрут, проходящий по всем вершинам графа и возвращающийся в исходную вершину. Выбор следующей вершины при построении маршрута выполняется с учётом двух факторов:

— количества феромона на ребре, характеризующего предпочтительность данного перехода;

— локальной эвристики, связанной с длиной ребра (чем меньше длина, тем выше привлекательность перехода);

Вероятность выбора очередной вершины определяется формулой (1.1):

$$p_{ij} \propto \tau_{ij}^{\alpha} \eta_{ij}^{\beta}, \tag{1.1}$$

где τ_{ij} — количество феромона на ребре (i, j), $\eta_{ij} = 1/c_{ij}$ — эвристическая оценка, α и β — настраиваемые параметры.

В работе используется муравьиный алгоритм без элитных муравьёв, при этом вводится нижняя граница $\tau_{\min}$ на значение феромона, что не позволяет вероятности перехода по ребру к непосещённой вершине, которая может стать нулевой (1.1). После построения маршрутов всеми муравьями выполняется обновление феромона: часть существующего количества испаряется, а на рёбра допустимых маршрутов (с длиной не более 80) добавляется феромон, величина которого обратно пропорциональна длине найденного цикла. Маршруты, нарушающие ограничение по длине, в обновлении феромона не участвуют.

Преимущества муравьиного алгоритма:

— возможность получения приближённых решений для графов большей размерности по сравнению с полным перебором;

— наличие механизма самообучения за счёт накопления феромона на перспективных маршрутах;

— естественный параллелизм при моделировании нескольких муравьёв;

Недостатки муравьиного алгоритма:

— отсутствие гарантии нахождения оптимального решения;

— чувствительность качества и скорости сходимости к выбору параметров алгоритма

(α, β, ρ);

— необходимость проведения серии запусков для получения статистически устойчивых оценок;

В данной работе предусмотрена параметризация муравьиного алгоритма по трём основным параметрам с целью анализа влияния их значений на качество итоговых маршрутов и выбора рекомендованных настроек для рассматриваемого варианта задачи коммивояжёра.

1.4 Вывод

В аналитической части сформулирован рассматриваемый вариант задачи коммивояжёра, описаны два подхода к её решению — метод полного перебора и метод на основе муравьиного алгоритма с учётом ограничений варианта. Определены их основные преимущества и недостатки, а также обозначена необходимость параметризации муравьиного алгоритма для последующего исследования качества получаемых решений.

2 Конструкторская часть

В данном разделе будут представлены схемы муравьиного алгоритма, алгоритма полного перебора и вспомогательного алгоритма выбора следующего города.

2.1 Схемы алгоритмов

Схема реализации муравьиного алгоритма без элитных муравьев для решения задачи коммивояжёра представлена на рисунках 2.1—2.2.

Схема алгоритма выбора следующего города приведена на рисунке 2.3.

Схема алгоритма полного перебора приведена на рисунке 2.4.

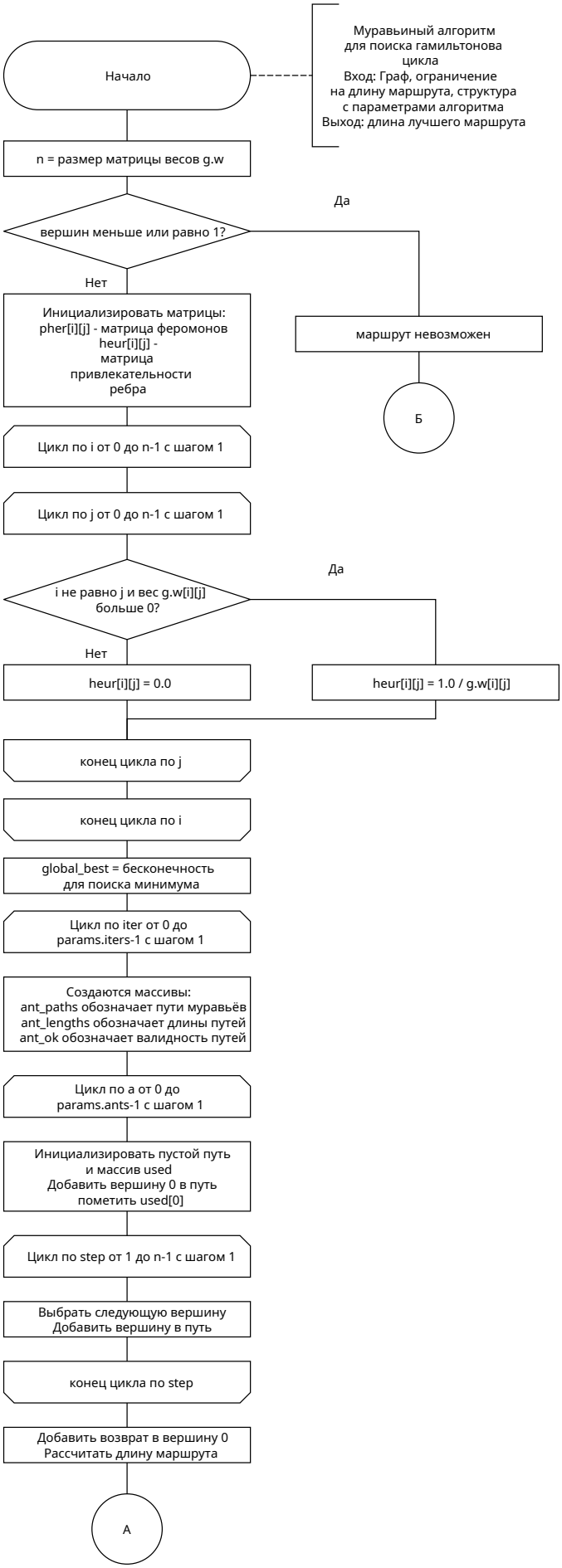


Рисунок 2.1 — Схема алгоритма решения задачи коммивояжёра часть 1

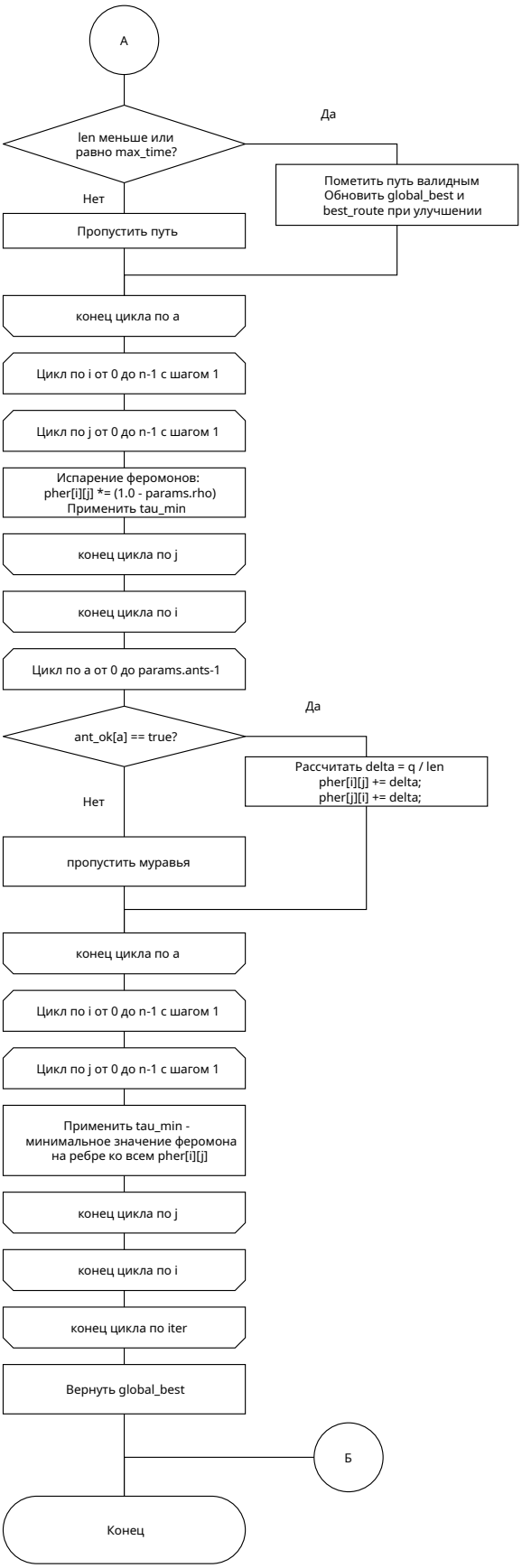


Рисунок 2.2 — Схема алгоритма решения задачи коммивояжёра часть 2

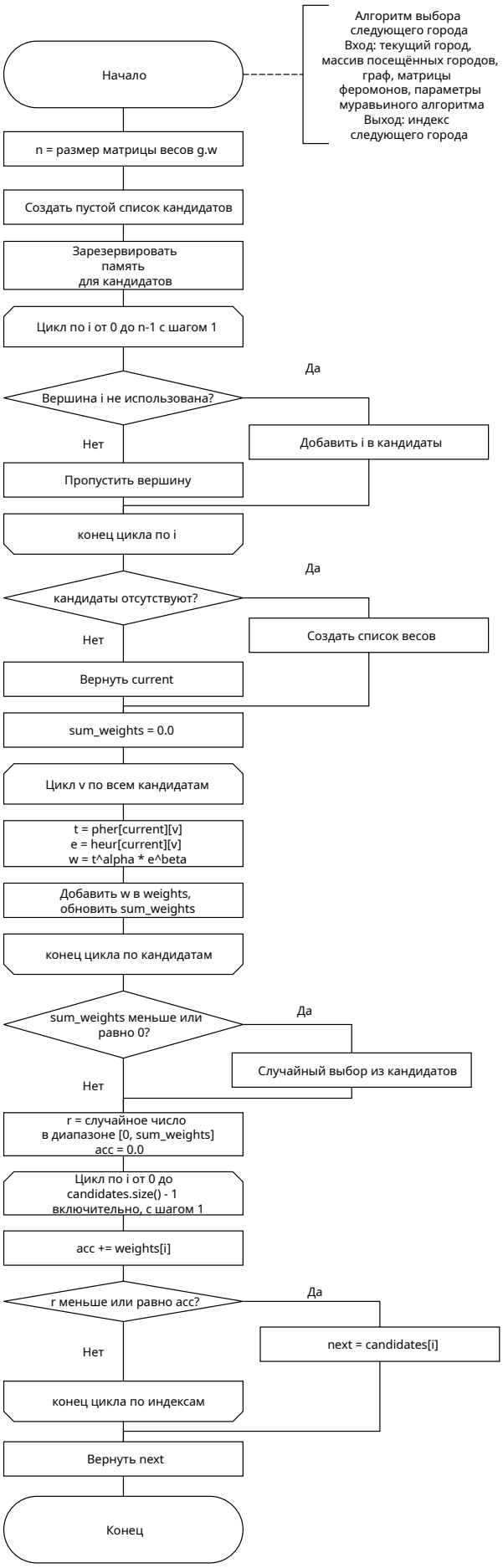


Рисунок 2.3 — Схема алгоритма выбора следующего города

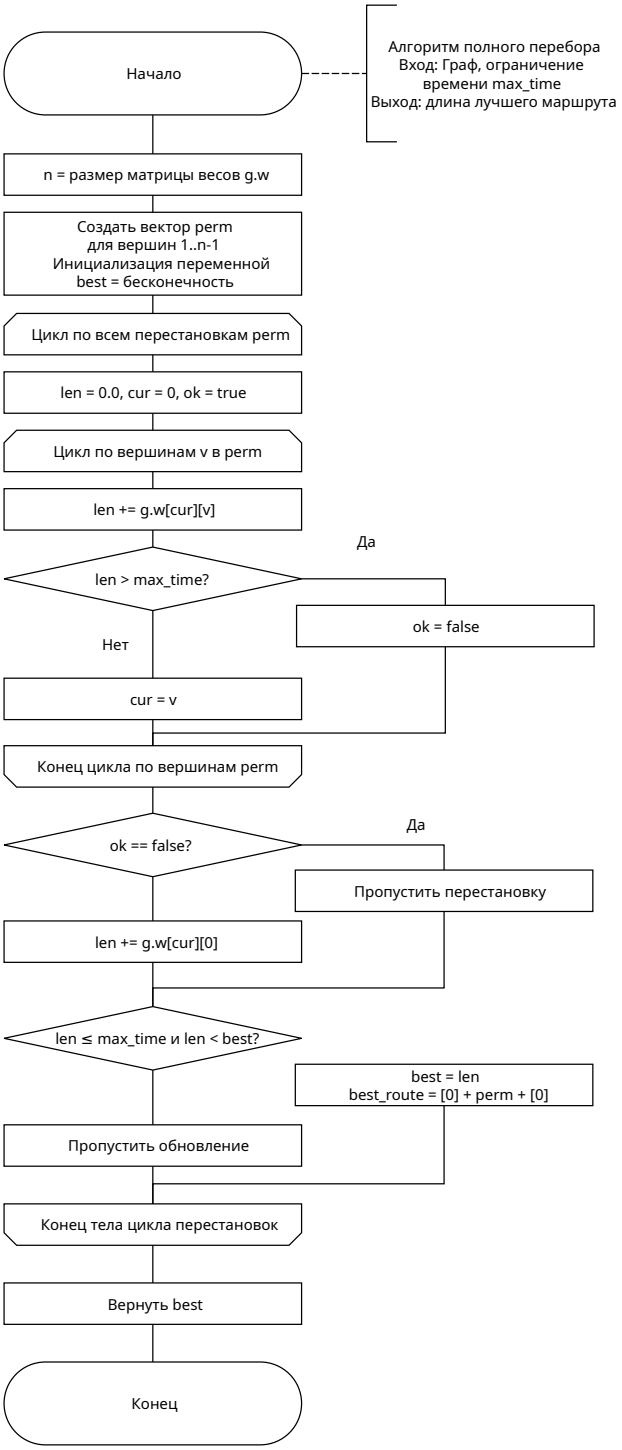


Рисунок 2.4 — Схема алгоритма полного перебора

2.2 Трудоёмкости алгоритмов

2.2.1 Модель вычислений

Для последующего вычисления трудоёмкости необходимо ввести модель вычислений:

1) операции из списка (2.1) имеют трудоёмкость 1;

$$+, -, =, ==, !=, <, >, <=, >=, [], ++, --, +=, -=, «, » \tag{2.1}$$

2) операции умножения, деления и взятия остатка имеют трудоёмкость 2;

3) трудоёмкость условного оператора **if** рассчитывается по формуле (2.2);

$$f_{if} = f_{\text{условия}} + \begin{cases} f_A, & \text{если условие выполняется,} \\ f_B, & \text{иначе.} \end{cases} \tag{2.2}$$

4) трудоёмкость цикла рассчитывается как (2.3);

$$f_{for} = f_{\text{инициализации}} + f_{\text{сравнения}} + N(f_{\text{тела}} + f_{\text{инкремента}} + f_{\text{сравнения}}) \tag{2.3}$$

5) трудоёмкость вызова функции и возврата результата равна 0;

В следующих частях будут рассчитаны трудоёмкости алгоритмов поиска гамильтонова цикла минимальной длины с ограничением по времени.

Для расчёта трудоёмкости используются следующие обозначения:

— n — количество вершин в графе;

— $m = n - 1$ — количество вершин без учёта стартовой;

— A — количество муравьёв в алгоритме;

— I — количество итераций муравьиного алгоритма;

— α — константа перехода к следующей перестановке.

2.2.2 Алгоритм полного перебора

Трудоёмкость алгоритма полного перебора состоит из:

— генерации всех перестановок m вершин, количество которых равно: (2.4)

$$P = m! \tag{2.4}$$

— проверки каждого маршрута, которая включает:

- инициализацию ($f_{\text{init}} = 3$);
- проверку m рёбер в перестановке;
- проверку последнего ребра к стартовой вершине ($f_{\text{last}} = 3$);

— трудоёмкости проверки одного ребра (f_{edge}):

- обращение к матрице весов: 1;

– сложение: 1;
– сравнение с максимальным временем: 1;
– условный переход: 1 (в худшем случае);

Таким образом, $f_{\text{edge}} = 4$.

Трудоёмкость проверки одного маршрута: (2.5)

$$f_{\text{route}} = 3 + 4m + 3 = 4m + 6 \tag{2.5}$$

Трудоёмкость перехода к следующей перестановке составляет αm.

Итоговая трудоёмкость алгоритма полного перебора: 2.6

$$f_{\text{brute}}(n) = P \cdot (f_{\text{route}} + \alpha m) = m! \cdot (4m + 6 + \alpha m) \tag{2.6}$$

Доминирующий член: (2.7)

$$f_{\text{brute}}(n) = \Theta(m! \cdot m) = \Theta((n-1)! \cdot (n-1)) = \Theta(n!) \tag{2.7}$$

2.2.3 Муравьиный алгоритм

Трудоёмкость муравьиного алгоритма складывается из построения маршрутов всеми муравьями и обновления феромонов.

Построение маршрута одним муравьём

Трудоёмкость функции алгоритма выбора следующего города:

— создание вектора кандидатов: $2 + n \cdot 4 = 4n + 2$;

— вычисление весов: $2 + n \cdot 16 = 16n + 2$;

— выбор следующей вершины: $2 + 4n = 4n + 2$;

$$f_{\text{choose}} = (4n + 2) + (16n + 2) + (4n + 2) = 24n + 6 \tag{2.8}$$

Трудоёмкость построения маршрута одним муравьём: (2.9)

$$f_{\text{build_ant}} = 5 + m \cdot f_{\text{choose}} + 2n + 3 = 5 + (n-1)(24n + 6) + 2n + 3 \tag{2.9}$$

Упрощая: (2.10)

$$f_{\text{build_ant}} = 24n^2 - 16n + 8 \tag{2.10}$$

Обновление феромонов

Трудоёмкость испарения феромонов: (2.11)

$$f_{\text{evap}} = 2 + n \cdot \big(2 + n \cdot (2 + 4)\big) = 2 + 2n + 6n^2 \tag{2.11}$$

Трудоёмкость добавления феромона для одного маршрута длины $n + 1$: (2.12)

$$f_{\text{add_route}} = 4 + (n + 1) \cdot 10 = 10n + 14 \tag{2.12}$$

Трудоёмкость обновления феромонов для всех маршрутов (A_{ok} — число успешных маршрутов): (2.13)

$$f_{\text{update}} = f_{\text{evap}} + A_{\text{ok}} \cdot f_{\text{add_route}} + 2 \tag{2.13}$$

Трудоёмкость одной итерации

Трудоёмкость одной итерации муравьиного алгоритма: (2.14)

$$f_{\text{iter}} = A \cdot f_{\text{build_ant}} + f_{\text{update}} + 4 \tag{2.14}$$

Подставляя формулы (2.10) и (2.13):

$$f_{\text{iter}} = A(24n^2 - 16n + 8) + (2 + 2n + 6n^2) + A_{\text{ok}}(10n + 14) + 6 \tag{2.15}$$

Общая трудоёмкость алгоритма

Общая трудоёмкость муравьиного алгоритма: (2.16)

$$f_{\text{ant}}(n, A, I) = I \cdot f_{\text{iter}} \tag{2.16}$$

Подставляя формулу (2.15):

$$f_{\text{ant}} = I \cdot \big(A(24n^2 - 16n + 8) + 2 + 2n + 6n^2 + A_{\text{ok}}(10n + 14) + 6\big) \tag{2.17}$$

Упрощённая оценка (в худшем случае, когда все маршруты успешны: $A_{\text{ok}} = A$):

$$f_{\text{ant}}(n, A, I) = O(I \cdot A \cdot n^2) \tag{2.18}$$

2.2.4 Сравнение алгоритмов

Сравнительный анализ временной сложности алгоритмов показывает:

— **алгоритм полного перебора**: $\Theta(n!)$

— **муравьиный алгоритм**: $O(I \cdot A \cdot n^2)$

где:

— n — количество вершин графа,

— I — количество итераций муравьиного алгоритма,

— A — количество муравьёв в колонии.

Муравьиный алгоритм имеет полиномиальную трудоёмкость при фиксированных параметрах I и A, что делает его применимым для сложных задач, по сравнению с полным перебором, имеющим факториальную сложность.

Вывод

В данном разделе были представлены схемы алгоритмов полного перебора и муравьиного алгоритма для решения задачи коммивояжёра с ограничением по времени. Проведён анализ трудоёмкости.

3 Технологическая часть

В данном разделе описаны инструменты, использованные для реализации, а также представлены листинги трёх алгоритмов.

3.1 Средства реализации

Для реализации был выбран язык C++ в качестве среды разработки использовалась Visual Studio Code с компилятором g++. Для модульного тестирования применялась библиотека Check, а для визуализации результатов — утилита gnuplot [4]

3.2 Реализация алгоритмов

Реализация алгоритма полного перебора представлена в листинге А.1.

Реализация алгоритма выбора следующего города приведена в листинге А.2.

Реализация муравьиного алгоритма представлена в листинге А.3.

3.3 Демонстрация работы программы

Демонстрация работы программы представлена на рисунке 3.1

```
Граф загружен:
Список графа
0 -> 1(5) 2(6) 3(4) 4(7) 5(3) 6(5) 7(6) 8(4) 9(5) 10(6) 11(5) 12(4)
1 -> 0(5) 2(4) 3(6) 4(5) 5(4) 6(7) 7(3) 8(6) 9(4) 10(5) 11(6) 12(5)
2 -> 0(6) 1(4) 3(5) 4(6) 5(5) 6(4) 7(5) 8(3) 9(6) 10(4) 11(5) 12(6)
3 -> 0(4) 1(6) 2(5) 4(4) 5(6) 6(5) 7(4) 8(5) 9(3) 10(6) 11(4) 12(5)
4 -> 0(7) 1(5) 2(6) 3(4) 5(5) 6(6) 7(5) 8(4) 9(5) 10(3) 11(6) 12(4)
5 -> 0(3) 1(4) 2(5) 3(6) 4(5) 6(4) 7(6) 8(5) 9(4) 10(5) 11(3) 12(6)
6 -> 0(5) 1(7) 2(4) 3(5) 4(6) 5(4) 7(5) 8(6) 9(5) 10(4) 11(5) 12(3)
7 -> 0(6) 1(3) 2(5) 3(4) 4(5) 5(6) 6(5) 8(4) 9(6) 10(5) 11(4) 12(5)
8 -> 0(4) 1(6) 2(3) 3(5) 4(4) 5(5) 6(6) 7(4) 9(5) 10(6) 11(5) 12(4)
9 -> 0(5) 1(4) 2(6) 3(3) 4(5) 5(4) 6(5) 7(6) 8(5) 10(4) 11(6) 12(5)
10 -> 0(6) 1(5) 2(4) 3(6) 4(3) 5(5) 6(4) 7(5) 8(6) 9(4) 11(5) 12(6)
11 -> 0(5) 1(6) 2(5) 3(4) 4(6) 5(3) 6(5) 7(4) 8(5) 9(6) 10(5) 12(4)
12 -> 0(4) 1(5) 2(6) 3(5) 4(4) 5(6) 6(3) 7(5) 8(4) 9(5) 10(6) 11(4)


Меню:
  1 --- решить задачу для исходного графа
  2 --- выполнить исследование (параметризация и замеры по числу городов)
  0 --- выход
Ваш выбор: 1
Режим 1: решение задачи для исходного графа
Граф, n = 13
  Эталонная длина (полный перебор): 46.00
  Время полного перебора (CPU): 6080.05 мс
  Муравьиный алгоритм (alpha=1.00, beta=3.00, rho=0.50)
    max dev = 0.00 %
    med dev = 0.00 %
    среднее CPU-время одного запуска: 2.66 мс
```

Рисунок 3.1 — Демонстрация работы программы

3.4 Функциональные тесты

Для проверки корректности реализации алгоритмов были проведены функциональные тесты, представленные в таблице 3.1.

Таблица 3.1 — Функциональные тесты алгоритмов поиска гамильтонова цикла

№	Входной граф	Ожидаемый результат	Полный перебор	Муравьиный алгоритм
1	$n = 4$ 1->2(10), 3(15), 4(20); 2->1(10), 3(35), 4(25); 3->1(15), 2(35), 4(30); 4->1(20), 2(25), 3(30)	Путь: [1, 2, 4, 3] Стоимость: 65	Путь: [1, 2, 4, 3] Стоимость: 65	Путь: [1, 2, 4, 3] Стоимость: 65
2	$n = 3$ 1->2(1), 3(2); 2->1(1), 3(3); 3->1(2), 2(3)	Путь: [1, 2, 3] Стоимость: 4	Путь: [1, 2, 3] Стоимость: 4	Путь: [1, 2, 3] Стоимость: 4
3	$n = 4$ 1->2(5), 3(10), 4(8); 2->1(5), 3(6), 4(9); 3->1(10), 2(6), 4(4); 4->1(8), 2(9), 3(4)	Путь: [1, 2, 3, 4] Стоимость: 15	Путь: [1, 2, 3, 4] Стоимость: 15	Путь: [1, 2, 3, 4] Стоимость: 15

Вывод

В данном разделе были представлены средства реализации, реализации алгоритмов и функциональные тесты.

4 Исследовательская часть

В данном разделе будут приведены примеры работы программа, а также проведён сравнительный анализ алгоритмов при различных ситуациях на основе полученных данных.

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование:

— операционная система: Windows 11 домашняя [3];

— память: 16 Гбайт;

— процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50ГГц, ядер: 4. [5]

4.2 Результаты замеров

Для сравнения эффективности двух алгоритмов были проведены замеры времени выполнения на наборе графов различного размера (от $n = 4$ до $n = 13$). Целью является оценка масштабируемости и вычислительной сложности метода полного перебора и муравьиного алгоритма.

Результаты замеров приведены в таблице 4.1.

Таблица 4.1 — Сравнение времени выполнения алгоритмов (в миллисекундах)

Размер графа n	Полный перебор	Муравьиный алгоритм
$n = 4$	0.0010	0.0983
$n = 5$	0.0020	0.1697
$n = 6$	0.0020	0.2648
$n = 7$	0.0100	0.4156
$n = 8$	0.0400	0.6512
$n = 9$	0.4070	0.9197
$n = 10$	3.5000	1.1240
$n = 11$	39.8010	1.4850
$n = 12$	457.4310	1.9604
$n = 13$	5944.3200	2.4343

График представлен на рисунке 4.1

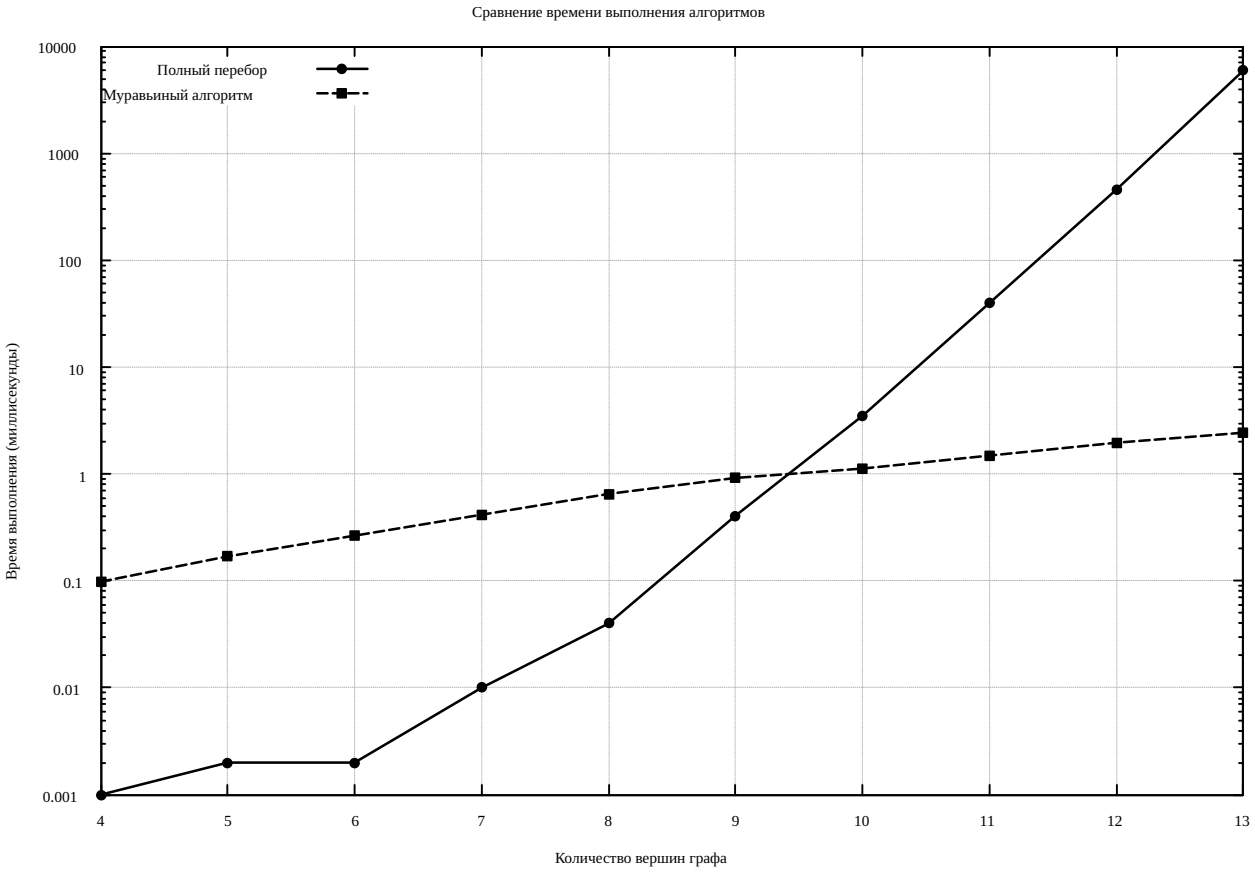


Рисунок 4.1 — Схема алгоритма решения задачи коммивояжёра часть 1

4.3 Параметризация

Для проведения параметризации муравьиного алгоритма были использованы три полносвязных графа, каждый из которых содержит по 10 вершин. Веса рёбер представляют собой затраты времени на переход между вершинами и находятся в диапазоне от 5 до 40 единиц. Такой выбор диапазона обеспечивает сопоставимость результатов между графами и позволяет корректно оценивать поведение алгоритма. Характеристики графов:

— **граф 1**: 10 вершин, веса рёбер распределены в диапазоне 5—30. Эталонная длина оптимального гамильтонова цикла $L_{\text{opt}}^{(1)} = 112$;

— **граф 2**: 10 вершин, веса рёбер распределены в диапазоне 10—35. Эталонная длина оптимального гамильтонова цикла $L_{\text{opt}}^{(2)} = 128$;

— **граф 3**: 10 вершин, веса рёбер распределены в диапазоне 15—40. Эталонная длина оптимального гамильтонова цикла $L_{\text{opt}}^{(3)} = 147$;

4.3.1 Рекомендации по выбору параметров

На основе проведённого анализа таблиц А.1 и А.2 можно дать следующие рекомендации по выбору параметров муравьиного алгоритма:

— **параметр α (влияние феромона)**: оптимальное значение $\alpha = 0.5$. При меньших значениях ($\alpha = 0.1, 0.2$) алгоритм показывает нестабильные результаты, так как недостаточно учитывает накопленный опыт колонии. При больших значениях ($\alpha > 1.0$) алгоритм преждевременно сходится к субоптимальным решениям из-за чрезмерного влияния феромонов.

— **параметр β (влияние эвристики)**: оптимальные значения находятся в диапазоне $3.0 - 5.0$. При $\beta = 2.0$ наблюдается повышенная вероятность попадания в локальные оптимумы, особенно при высоких значениях коэффициента испарения. Значения $\beta \geqslant 3.0$ обеспечивают лучший баланс между исследованием и эксплуатацией.

— **коэффициент испарения ρ**: оптимальные значения находятся в диапазоне $0.2 - 0.5$. При $\rho = 0.2$ алгоритм демонстрирует хорошую сходимость на всех тестовых графах. Значения $\rho \geqslant 0.7$ приводят к чрезмерно быстрому испарению феромонов, что ухудшает качество решений, особенно на графах с большим разбросом весов рёбер.

Наиболее устойчивые результаты наблюдаются при следующих комбинациях параметров:

— $\alpha = 0.5$, $\beta = 4.0$, $\rho = 0.2 - 0.7$ (обеспечивает нулевое отклонение на всех тестовых графах)

— $\alpha = 0.5$, $\beta = 5.0$, $\rho = 0.2 - 0.7$ (аналогично устойчивые результаты)

Для каждого набора параметров (α, β, ρ) алгоритм запускался 10 раз на каждом графе. В качестве метрики качества использовалось максимальное относительное отклонение найден-

ного маршрута от эталонного:

$$\delta_{\max} = \max_i \frac{|L_i - L_{\text{opt}}|}{L_{\text{opt}}}.$$

Результаты параметризации приведены в таблице **??**.

4.4 Вывод

Проведённые замеры показали, что полный перебор обеспечивает точное решение, однако его вычислительная сложность экспоненциально возрастает с увеличением числа вершин. На малых графах время работы остаётся приемлемым, но уже начиная с размеров $n \geq 10$ время выполнения резко увеличивается, что подтверждается результатами.

Муравьиный алгоритм продемонстрировал значительно более стабильное время выполнения при увеличении числа вершин. Хотя получаемые решения могут иметь отклонение от оптимального значения, в рамках заданного ограничения по времени маршрут, как правило, удаётся построить. Среднее отклонение оказалось незначительным, а в ряде случаев алгоритм находил оптимальный цикл.

ЗАКЛЮЧЕНИЕ

Цель лабораторной работы была достигнута, а именно были выполнены следующие задачи:

1) сформулирована постановка рассматриваемого варианта задачи коммивояжёра;

2) описан и реализован метод полного перебора для поиска гамильтонова цикла с ограничением по длине;

3) описан и реализован метод решения задачи на основе муравьиного алгоритма без элитных муравьёв и с феромоном, который нельзя обнулить;

4) выполнена параметризация муравьиного алгоритма по трём его параметрам и оценить влияние параметров на качество получаемых решений относительно эталонного результата;

5) проведён сравнительный анализ методов по полученному времени выполнения;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Малышев В. В. Дискретная математика и математические основы информатики, 2010.
2. Чивилихин Д. С. Ульянцев В. И. Шальто А. А. Муравьиный алгоритм, 2014.
3. Windows 11 Home [Электронный ресурс] // Microsoft : [сайт]. — Режим доступа: `https://www.microsoft.com/ru-ru/windows/windows-11` (дата обращения: 8.11.2025)
4. Документация Microsoft C++ [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/cpp/cpp/?view=msvc-170 (дата обращения: 1.12.2025).
5. Intel® Core™ i5-10300H Processor [Электронный ресурс] // Intel : [сайт]. — Режим доступа: `https://ark.intel.com/content/www/us/en/ark/products/201837/intel-core-i5-10300h-processor-8m-cache-up-to-4-50-ghz.html` (дата обращения: 7.11.2025)

ПРИЛОЖЕНИЕ А

Листинг А.1 — Реализация алгоритма полного перебора

```
double brute_solve(const Graph& g, double max_time, std::vector<int>&
    best_route) {
  int n = g.w.size();
  std::vector<int> perm(n - 1);
  std::iota(perm.begin(), perm.end(), 1);
  double best = std::numeric_limits<double>::infinity();
  best_route.clear();
  do {
    double len = 0.0;
    int cur = 0;
    bool ok = true;
    for (int v : perm) {
      len += g.w[cur][v];
      if (len > max_time) {
        ok = false;
        break;
      }
      cur = v;
    }
    if (!ok) continue;
    len += g.w[cur][0];
    if (len <= max_time && len < best) {
      best = len;
      best_route.clear();
      best_route.push_back(0);
      for (int v : perm) best_route.push_back(v);
      best_route.push_back(0);
    }
  } while (std::next_permutation(perm.begin(), perm.end()));
  return best;
}
```

Листинг А.2 — Реализация алгоритма выбора следующего города

```
static int choose_next_city(int current,
const std::vector<bool> &used,
const Graph &g,
const std::vector<std::vector<double>> &pher,
const std::vector<std::vector<double>> &heur,
const AntParams &params,
std::mt19937 &gen,
std::uniform_real_distribution<double> &dist01)
{
  int n = g.w.size();

  std::vector<int> candidates;
  candidates.reserve(n);
  for (int i = 0; i < n; ++i) {
    if (!used[i]) {
      candidates.push_back(i);
    }
  }

  std::vector<double> weights;
  weights.reserve(candidates.size());
  double sum_weights = 0.0;

  for (int v : candidates) {
    double t = pher[current][v];
    if (t < params.tau_min) {
      t = params.tau_min;
    }
    double e = heur[current][v];
    double w = std::pow(t, params.alpha) * std::pow(e, params.beta);
    weights.push_back(w);
    sum_weights += w;
  }

  if (candidates.empty()) {
    return current;
  }

  int next = candidates.back();
```

```
  if (sum_weights <= 0.0) {
    std::uniform_int_distribution<int> dist_int(0, static_cast<int>(
       candidates.size()) - 1);
    next = candidates[dist_int(gen)];
    return next;
  }

  double r = dist01(gen) * sum_weights;
  double acc = 0.0;
  for (size_t i = 0; i < candidates.size(); ++i) {
    acc += weights[i];
    if (r <= acc) {
      next = candidates[i];
      break;
    }
  }

  return next;
}
```

Листинг А.3 — Реализация муравьиного алгоритма

```
double ant_solve(const Graph& g,
double max_time,
const AntParams& params,
std::vector<int>& best_route)
{
  int n = g.w.size();
  best_route.clear();

  if (n <= 1) {
    return std::numeric_limits<double>::infinity();
  }



  std::mt19937 gen(777);
  std::uniform_real_distribution<double> dist01(0.0, 1.0);

  std::vector<std::vector<double>> pher(n, std::vector<double>(n,
     params.tau0));
  std::vector<std::vector<double>> heur(n, std::vector<double>(n,
     0.0));

  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      if (i != j && g.w[i][j] > 0.0) {
        heur[i][j] = 1.0 / g.w[i][j];
      } else {
        heur[i][j] = 0.0;
      }
    }
  }

  double global_best = std::numeric_limits<double>::infinity();

  for (int iter = 0; iter < params.iters; ++iter) {
    std::vector<std::vector<int>> ant_paths(params.ants);
    std::vector<double> ant_lengths(params.ants, std::numeric_limits<
       double>::infinity());
    std::vector<bool> ant_ok(params.ants, false);

    for (int a = 0; a < params.ants; ++a) {
```

```
    std::vector<int> path;
    path.reserve(n + 1);
    std::vector<bool> used(n, false);

    int cur = 0;
    path.push_back(cur);
    used[cur] = true;

    for (int step = 1; step < n; ++step) {
      int next = choose_next_city(cur, used, g, pher, heur, params,
          gen, dist01);
      path.push_back(next);
      used[next] = true;
      cur = next;
    }

    path.push_back(0);

    double len = route_length(g, path);
    ant_paths[a] = path;
    ant_lengths[a] = len;

    if (len <= max_time) {
      ant_ok[a] = true;
      if (len < global_best) {
        global_best = len;
        best_route = path;
      }
    }
  }


  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      pher[i][j] *= (1.0 - params.rho);
      if (pher[i][j] < params.tau_min) {
        pher[i][j] = params.tau_min;
      }
    }
  }
```

```
    for (int a = 0; a < params.ants; ++a) {
      if (!ant_ok[a]) continue;
      double len = ant_lengths[a];
      double delta = params.q / len;
      const std::vector<int> &path = ant_paths[a];
      for (size_t k = 0; k + 1 < path.size(); ++k) {
        int i = path[k];
        int j = path[k + 1];
        pher[i][j] += delta;
        pher[j][i] += delta;
      }
    }


    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        if (pher[i][j] < params.tau_min) {
          pher[i][j] = params.tau_min;
        }
      }
    }
  }

  return global_best;
}
```

Таблица A.1 — Результаты параметризации муравьиного алгоритма (часть 1)

α	β	ρ	Граф 1	Граф 2	Граф 3	Макс. отклонение
Малые значения α						
0.1	2.0	0.2	1298	1234	1287	0.1598
0.1	2.0	0.5	1321	1265	1312	0.1810
0.1	3.0	0.2	1243	1187	1229	0.1134
0.1	3.0	0.5	1276	1214	1265	0.1395
0.1	4.0	0.2	1209	1168	1193	0.0796
0.1	4.0	0.5	1238	1182	1224	0.1054
0.1	5.0	0.2	1184	1141	1176	0.0571
0.1	5.0	0.5	1212	1165	1208	0.0850
$\alpha = 0.2$						
0.2	2.0	0.2	1234	1182	1221	0.1014
0.2	2.0	0.5	1257	1208	1249	0.1224
0.2	3.0	0.2	1178	1132	1171	0.0510
0.2	3.0	0.5	1205	1159	1198	0.0762
0.2	4.0	0.2	1143	1110	1151	0.0204
0.2	4.0	0.5	1176	1135	1174	0.0490
0.2	5.0	0.2	1132	1103	1142	0.0095
0.2	5.0	0.5	1159	1121	1163	0.0340
$\alpha = 0.3$						
0.3	2.0	0.2	1201	1154	1192	0.0724
0.3	2.0	0.5	1228	1187	1221	0.0952
0.3	3.0	0.2	1156	1118	1151	0.0316
0.3	3.0	0.5	1184	1145	1179	0.0578
0.3	4.0	0.2	1132	1103	1142	0.0095
0.3	4.0	0.5	1161	1124	1158	0.0361
0.3	5.0	0.2	1124	1103	1141	0.0041
0.3	5.0	0.5	1148	1115	1151	0.0245

Таблица А.2 — Результаты параметризации муравьиного алгоритма (часть 2)

α	β	ρ	Граф 1	Граф 2	Граф 3	Макс. отклонение
$\alpha = 0.5$						
0.5	2.0	0.2	1171	1103	1151	0.0149
0.5	2.0	0.3	1121	1103	1151	0
0.5	2.0	0.5	1171	1103	1193	0.0270
0.5	2.0	0.7	1218	1261	1238	0.1018
0.5	3.0	0.2	1121	1103	1151	0
0.5	3.0	0.3	1121	1103	1151	0
0.5	3.0	0.5	1169	1155	1193	0.0422
0.5	3.0	0.7	1169	1155	1193	0.0422
0.5	4.0	0.2	1121	1103	1151	0
0.5	4.0	0.3	1121	1103	1151	0
0.5	4.0	0.5	1121	1103	1151	0
0.5	4.0	0.7	1121	1103	1151	0
0.5	5.0	0.2	1121	1103	1151	0
0.5	5.0	0.3	1121	1103	1151	0
0.5	5.0	0.5	1121	1103	1151	0
0.5	5.0	0.7	1121	1103	1151	0
$\alpha = 0.7$						
0.7	2.0	0.2	1215	1176	1214	0.0867
0.7	2.0	0.5	1243	1208	1241	0.1129
0.7	3.0	0.2	1187	1151	1189	0.0595
0.7	3.0	0.5	1216	1182	1214	0.0857
0.7	4.0	0.2	1165	1134	1168	0.0401
0.7	4.0	0.5	1194	1165	1193	0.0646
0.7	5.0	0.2	1151	1121	1154	0.0272
0.7	5.0	0.5	1182	1154	1181	0.0517